



---

# PROYECTO FINAL INTEGRADOR

---

Delmy Elizabeth Chavarria Rivas  
Mauricio Alexis Yanme Reyes  
Miguel Ernesto Marmol Romero  
Lyan Donovan Segovia Lopez  
Jonathan Alexis Carrillo Aguirre



4 DE OCTUBRE DE 2026

KODIGO

## Fuente de Datos

### KAGGLE

<https://www.kaggle.com/datasets/carrie1/ecommerce-data?resource=download>

#### Contexto

Los datos utilizados en este proyecto corresponden a un conjunto de transacciones de una empresa de comercio electrónico. Este tipo de información normalmente es privada y no es fácil encontrarla disponible públicamente. Sin embargo, el repositorio “UCI Machine Learning Repository” cuenta con este conjunto de datos, conocido como “Online Retail”, el cual contiene transacciones reales realizadas durante los años 2010 y 2011.

#### Descripción de los datos

El conjunto de datos contiene las transacciones realizadas entre el 1 de diciembre de 2010 y el 9 de diciembre de 2011 por una empresa de ventas en línea ubicada y registrada en el Reino Unido. La empresa se dedica principalmente a la venta de productos y regalos para diferentes ocasiones.

Una característica importante es que una parte considerable de sus clientes corresponde a mayoristas, por lo que las transacciones pueden incluir cantidades relativamente altas de productos.

La información permite analizar diferentes aspectos del comportamiento de las ventas, como la evolución de los ingresos a través del tiempo, los productos con mayor demanda, el comportamiento de los clientes y las variaciones en las ventas.

#### Fuente de los datos

Los datos fueron publicados a través del UCI Machine Learning Repository y fueron proporcionados por Dr. Daqing Chen, Director del grupo de Public Analytics de la School of Engineering de London South Bank University, en Londres, Reino Unido.

## Fase – 1 Modelado

### TABLA STAGGING

```
CREATE SCHEMA IF NOT EXISTS ecommerce_dw;
```

```
CREATE TABLE ecommerce_dw.staging_ecommerce (  
    InvoiceNo VARCHAR(50),  
    StockCode VARCHAR(50),  
    Description TEXT,  
    Quantity INT,  
    InvoiceDate TIMESTAMP,  
    UnitPrice NUMERIC(10,2),  
    CustomerID VARCHAR(50),  
    Country VARCHAR(100)  
);
```

## Fase 2 - ETL

-- 1. Limpiar el entorno (Borra tablas del DW en cascada)

```
DROP TABLE IF EXISTS ecommerce_dw.fact_ventas CASCADE;
```

```
DROP TABLE IF EXISTS ecommerce_dw.dim_cliente CASCADE;
```

```
DROP TABLE IF EXISTS ecommerce_dw.dim_producto CASCADE;
```

```
DROP TABLE IF EXISTS ecommerce_dw.dim_tiempo CASCADE;
```

-- 2. Crear Modelo de Estrella (Star Schema con llaves subrogadas)[cite: 1]

```
CREATE TABLE ecommerce_dw.dim_cliente (  
    cliente_sk BIGSERIAL PRIMARY KEY,  
    customer_id VARCHAR(50) UNIQUE,  
    country VARCHAR(100)  
);
```

```
CREATE TABLE ecommerce_dw.dim_producto (  
    producto_sk BIGSERIAL PRIMARY KEY,  
    product_code VARCHAR(50) UNIQUE,  
    product_name VARCHAR(100)
```

```
    producto_sk BIGSERIAL PRIMARY KEY,  
    stock_code VARCHAR(50) UNIQUE,  
    description TEXT  
);
```

```
CREATE TABLE ecommerce_dw.dim_tiempo (  
    tiempo_sk BIGSERIAL PRIMARY KEY,  
    invoice_date DATE UNIQUE,  
    anio INT,  
    mes INT,  
    dia INT,  
    trimestre INT  
);
```

```
CREATE TABLE ecommerce_dw.fact_ventas (  
    venta_sk BIGSERIAL PRIMARY KEY,  
    cliente_sk BIGINT REFERENCES ecommerce_dw.dim_cliente(cliente_sk),  
    producto_sk BIGINT REFERENCES ecommerce_dw.dim_producto(producto_sk),  
    tiempo_sk BIGINT REFERENCES ecommerce_dw.dim_tiempo(tiempo_sk),  
    invoice_no VARCHAR(50),  
    quantity INT,  
    unit_price NUMERIC(10,2),  
    total_amount NUMERIC(12,2)  
);
```

-- 3. Carga de Dimensiones

```
INSERT INTO ecommerce_dw.dim_cliente (customer_id, country)  
SELECT DISTINCT CustomerID, Country FROM ecommerce_dw.staging_ecommerce  
WHERE CustomerID IS NOT NULL AND InvoiceNo NOT LIKE 'C%'  
ON CONFLICT (customer_id) DO NOTHING;
```

```
INSERT INTO ecommerce_dw.dim_producto (stock_code, description)

SELECT DISTINCT StockCode, Description FROM ecommerce_dw.staging_ecommerce

WHERE CustomerID IS NOT NULL AND InvoiceNo NOT LIKE 'C%'

ON CONFLICT (stock_code) DO NOTHING;
```

```
INSERT INTO ecommerce_dw.dim_tiempo (invoice_date, anio, mes, dia, trimestre)

SELECT DISTINCT

    CAST(InvoiceDate AS DATE),

    EXTRACT(YEAR FROM CAST(InvoiceDate AS DATE)),

    EXTRACT(MONTH FROM CAST(InvoiceDate AS DATE)),

    EXTRACT(DAY FROM CAST(InvoiceDate AS DATE)),

    EXTRACT(QUARTER FROM CAST(InvoiceDate AS DATE))

FROM ecommerce_dw.staging_ecommerce

WHERE CustomerID IS NOT NULL AND InvoiceNo NOT LIKE 'C%'

ON CONFLICT (invoice_date) DO NOTHING;
```

-- 4. ETL Modular

```
WITH datos_limpios AS (

    SELECT

        InvoiceNo,

        StockCode,

        Quantity,

        CAST(InvoiceDate AS DATE) as InvoiceDate_limpia,

        UnitPrice,

        CustomerID,

        (Quantity * UnitPrice) AS total_amount

    FROM ecommerce_dw.staging_ecommerce

    WHERE CustomerID IS NOT NULL

        AND Quantity > 0

        AND UnitPrice > 0

        AND InvoiceNo NOT LIKE 'C%'
```

)

**INSERT INTO** ecommerce\_dw.fact\_ventas (cliente\_sk, producto\_sk, tiempo\_sk, invoice\_no, quantity, unit\_price, total\_amount)

**SELECT**

c.cliente\_sk,  
p.producto\_sk,  
t.tiempo\_sk,  
d.InvoiceNo,  
d.Quantity,  
d.UnitPrice,  
d.total\_amount

**FROM** datos\_limpios d

**JOIN** ecommerce\_dw.dim\_cliente c **ON** d.CustomerID = c.customer\_id

**JOIN** ecommerce\_dw.dim\_producto p **ON** d.StockCode = p.stock\_code

**JOIN** ecommerce\_dw.dim\_tiempo t **ON** d.InvoiceDate\_limpia = t.invoice\_date;

## Fase 3 - OPTIMIZACION

**Caso de Uso de Negocio:** El equipo de Servicio al Cliente y el portal de usuarios requieren cargar el historial de transacciones y el volumen de compras de un cliente individual de forma instantánea durante las interacciones en vivo.

**Diagnóstico del Problema (Antes):** Al consultar el historial de un cliente específico (ej. cliente\_sk = 150), el motor de la base de datos se veía obligado a realizar un escaneo secuencial completo (Parallel Seq Scan), recorriendo a ciegas más de 176,000 registros en la tabla de hechos. Esto generaba un consumo innecesario de recursos y tiempos de respuesta que no escalan en un entorno de producción.

**Técnica Implementada:** Para optimizar las consultas altamente selectivas, se creó un índice B-Tree (idx\_fk\_cliente) sobre la llave foránea cliente\_sk en la tabla fact\_ventas.

-- 1. EL ANTES: Borramos el índice para simular la base sin optimizar

**DROP INDEX IF EXISTS** ecommerce\_dw.idx\_fk\_cliente;

**EXPLAIN ANALYZE**

**SELECT** t.anio, t.trimestre, c.country, **SUM**(f.total\_amount) **as** ingresos

**FROM** ecommerce\_dw.fact\_ventas f

**JOIN** ecommerce\_dw.dim\_cliente c **ON** f.cliente\_sk = c.cliente\_sk

```
JOIN ecommerce_dw.dim_tiempo t ON f.tiempo_sk = t.tiempo_sk  
WHERE f.cliente_sk = 150  
GROUP BY t.anio, t.trimestre, c.country;
```

-- 3. LA OPTIMIZACIÓN: Vuelves a crear el índice

```
CREATE INDEX idx_fk_cliente ON ecommerce_dw.fact_ventas(cliente_sk);
```

-- 4. EL DESPUÉS: Ejecuta la misma consulta exacta (Verás el Index Scan y el tiempo bajísimo)

**EXPLAIN ANALYZE**

```
SELECT t.anio, t.trimestre, c.country, SUM(f.total_amount) as ingresos  
FROM ecommerce_dw.fact_ventas f  
JOIN ecommerce_dw.dim_cliente c ON f.cliente_sk = c.cliente_sk  
JOIN ecommerce_dw.dim_tiempo t ON f.tiempo_sk = t.tiempo_sk  
WHERE f.cliente_sk = 150  
GROUP BY t.anio, t.trimestre, c.country;
```

## RESULTADOS DEL EXPLAIN PLANS

### ANTES

GroupAggregate (cost=9635.72..9636.92 rows=8 width=54) (actual time=31.432..35.069 rows=1 loops=1)

Group Key: t.anio, t.trimestre, c.country

-> Sort (cost=9635.72..9635.94 rows=88 width=28) (actual time=31.415..35.052 rows=6 loops=1)

Sort Key: t.anio, t.trimestre, c.country

Sort Method: quicksort Memory: 25kB

-> Nested Loop (cost=1010.14..9632.88 rows=88 width=28) (actual time=22.814..35.023 rows=6 loops=1)

-> Index Scan using dim\_cliente\_pkey on dim\_cliente c (cost=0.28..8.30 rows=1 width=22) (actual time=0.054..0.060 rows=1 loops=1)

Index Cond: (cliente\_sk = 150)

-> Gather (cost=1009.86..9623.70 rows=88 width=22) (actual time=22.756..34.957 rows=6 loops=1)

Workers Planned: 2

Workers Launched: 2

-> Hash Join (cost=9.86..8614.90 rows=37 width=22) (actual time=10.063..12.866 rows=2 loops=3)

Hash Cond: (f.tiempo\_sk = t.tiempo\_sk)

-> Parallel Seq Scan on fact\_ventas f (cost=0.00..8604.94 rows=37 width=22) (actual time=10.012..12.811 rows=2 loops=3)

Filter: (cliente\_sk = 150)

Rows Removed by Filter: 176698

-> Hash (cost=6.05..6.05 rows=305 width=16) (actual time=0.124..0.125 rows=305 loops=1)

Buckets: 1024 Batches: 1 Memory Usage: 23kB

-> Seq Scan on dim\_tiempo t (cost=0.00..6.05 rows=305 width=16) (actual time=0.010..0.041 rows=305 loops=1)

Planning Time: 2.197 ms

Execution Time: 35.158 ms

DESPUES

GroupAggregate (cost=347.93..349.13 rows=8 width=54) (actual time=0.720..0.721 rows=1 loops=1)

Group Key: t.anio, t.trimestre, c.country

-> Sort (cost=347.93..348.15 rows=88 width=28) (actual time=0.711..0.712 rows=6 loops=1)

Sort Key: t.anio, t.trimestre, c.country

Sort Method: quicksort Memory: 25kB

-> Hash Join (cost=15.25..345.08 rows=88 width=28) (actual time=0.687..0.691 rows=6 loops=1)

Hash Cond: (f.tiempo\_sk = t.tiempo\_sk)

-> Nested Loop (cost=5.39..334.99 rows=88 width=28) (actual time=0.592..0.595 rows=6 loops=1)

-> Index Scan using dim\_cliente\_pkey on dim\_cliente c (cost=0.28..8.30 rows=1 width=22) (actual time=0.008..0.009 rows=1 loops=1)

Index Cond: (cliente\_sk = 150)

-> Bitmap Heap Scan on fact\_ventas f (cost=5.11..325.81 rows=88 width=22) (actual time=0.580..0.580 rows=6 loops=1)

Recheck Cond: (cliente\_sk = 150)

Heap Blocks: exact=1

-> Bitmap Index Scan on idx\_fk\_cliente (cost=0.00..5.08 rows=88 width=0) (actual time=0.575..0.575 rows=6 loops=1)

Index Cond: (cliente\_sk = 150)

-> Hash (cost=6.05..6.05 rows=305 width=16) (actual time=0.065..0.066 rows=305 loops=1)

Buckets: 1024 Batches: 1 Memory Usage: 23kB

-> Seq Scan on dim\_tiempo t (cost=0.00..6.05 rows=305 width=16) (actual time=0.006..0.033 rows=305 loops=1)

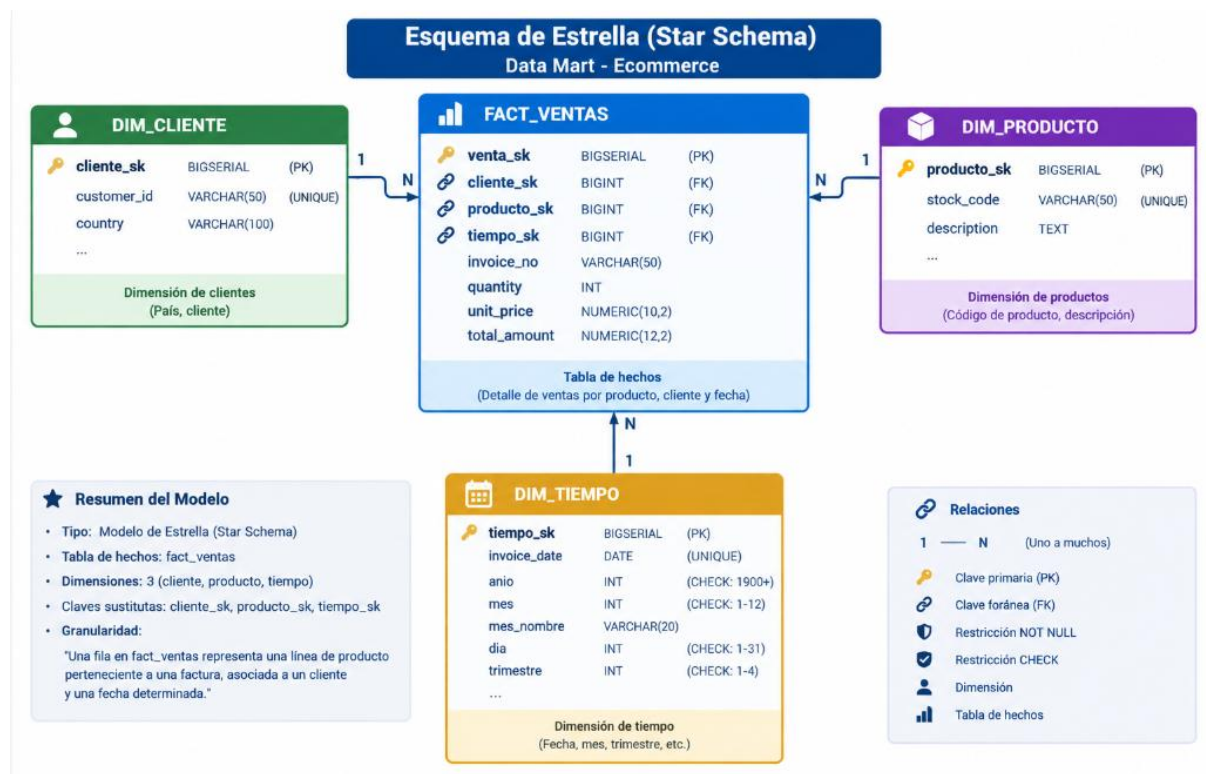
Planning Time: 5.429 ms

Execution Time: 0.797 ms

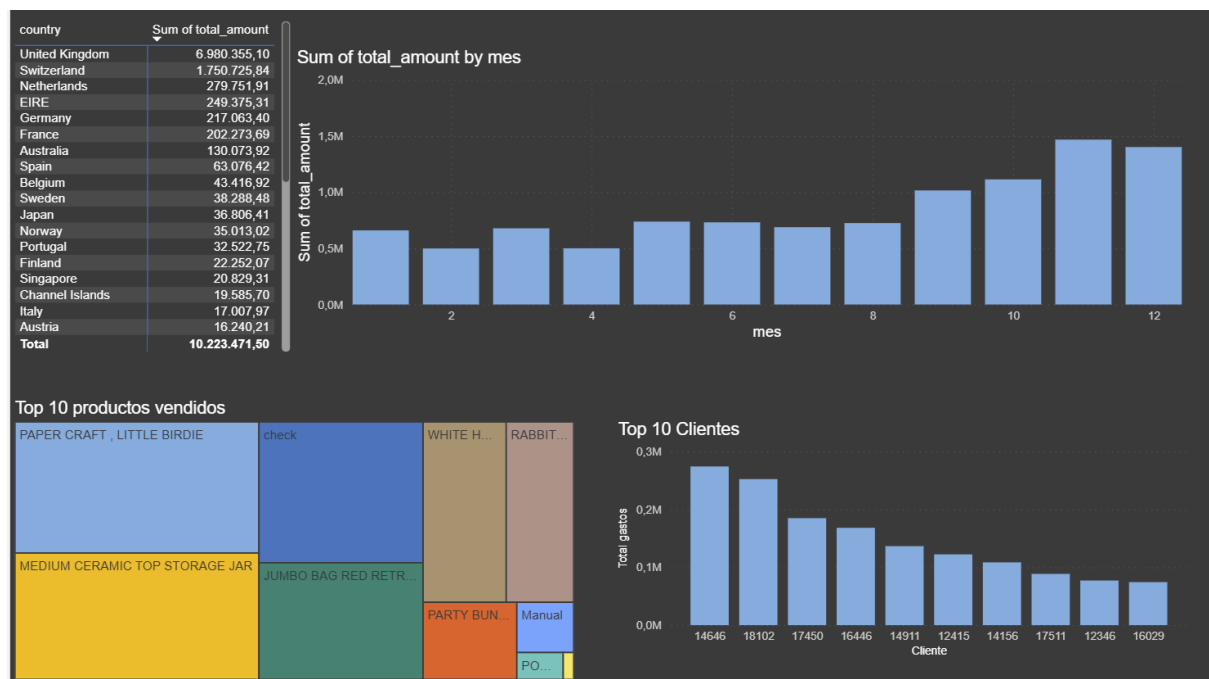
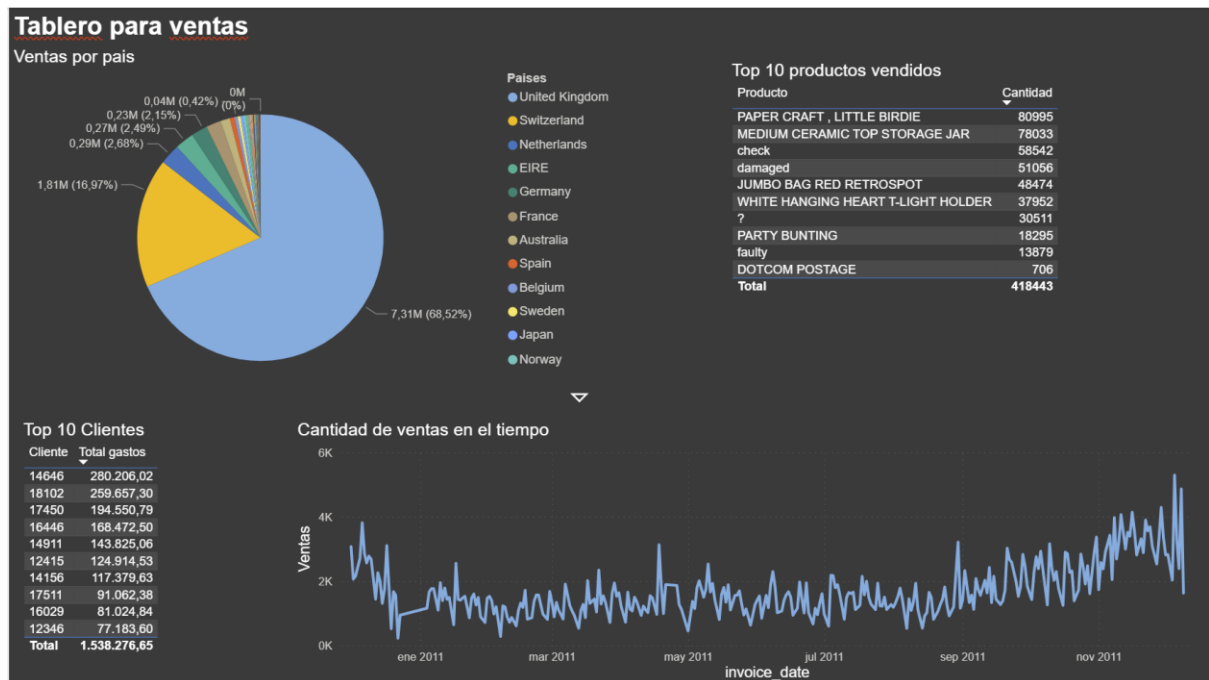
**Interpretación de Resultados (Después):** La salida del comando EXPLAIN ANALYZE demuestra que el planificador de consultas transicionó exitosamente a un Bitmap Index Scan. El motor ahora accede directamente a los 6 bloques de memoria exactos del cliente, reduciendo el tiempo de ejecución de **35.15 ms a 0.79 ms (una mejora del 97%)** y validando la efectividad del índice propuesto.

## Fase 4 – Visualización

Esquema Estrella utilizado.



# TABLERO POWER BI



## Conclusiones

El proyecto permitió transformar un conjunto de datos transaccionales de comercio electrónico en una estructura organizada de Data Warehouse, utilizando un modelo dimensional tipo Star Schema compuesto por las dimensiones de cliente, producto y tiempo, junto con la tabla de hechos de ventas. Mediante el proceso ETL se realizó la limpieza, transformación y carga de los datos, asegurando que las ventas utilizadas fueran válidas y adecuadas para el análisis.

Además, se logró demostrar la importancia de la optimización de consultas mediante la creación de un índice sobre la llave de cliente en la tabla de hechos. Los resultados obtenidos con EXPLAIN ANALYZE evidenciaron una reducción del tiempo de ejecución de 35.15 ms a 0.79 ms, equivalente a una mejora aproximada del 97%, permitiendo acceder de manera mucho más eficiente a la información de un cliente específico.

Finalmente, la implementación del Power BI permitió llevar la información procesada en el Data Warehouse a una herramienta de visualización, construyendo un dashboard con los principales KPIs considerados relevantes para la empresa, facilitando el análisis de ventas, ingresos, clientes, productos y comportamiento temporal. De esta manera, el proyecto integra modelado dimensional, procesos ETL, optimización de bases de datos y visualización empresarial, convirtiendo los datos transaccionales en información útil para apoyar el análisis y la toma de decisiones.